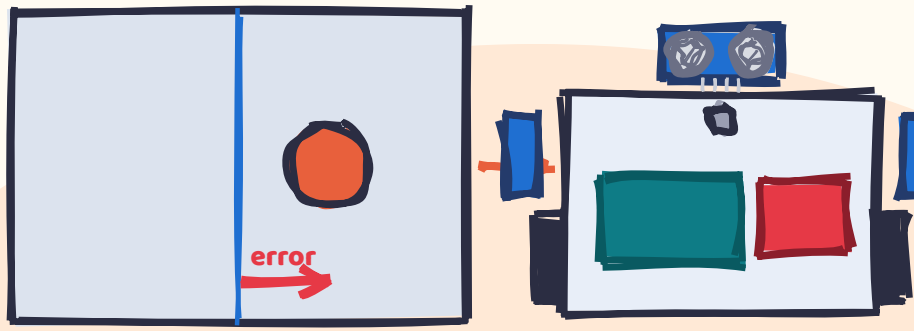
Tech Leaders · Class 18

Vision-Guided Robot Car

Your laptop sees. Your car follows.

Closed loop | P-control | UDP | Fail safe



Did you know?
Self-driving cars run their sense-decide-act loop many times every second, using several cameras plus radar and lidar.

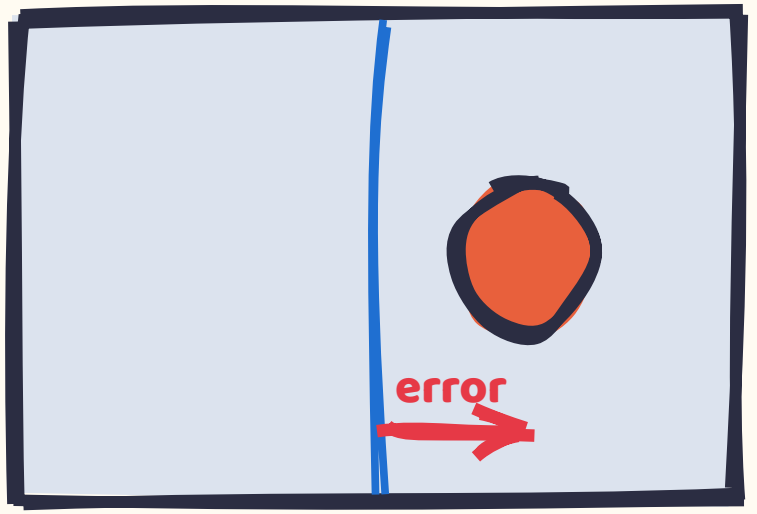
LEARN

Let's think...

How do you steer towards a target?

Follow the ball

Last class your tracker found an orange ball. Now connect it to wheels. If the ball is to the right, turn right. But how much? Turn too little and you lose it. Too much and you zig-zag like a confused bee.



Turn fully!

Guess?

Turn by how far off it is!

Today: you build a car that **follows a coloured object**: OpenCV on the laptop, commands over Wi-Fi, motors on an ESP32.

LEARN

Learning objectives

By the end of this class I can...

1	**Closed loop:** describe a **sense** → **decide** → **act** loop and calculate the steering **error**	**Proof:** I can show the error number change as the object moves
2	**P-controller:** steer with a **proportional controller** and tune **Kp** to avoid wobbling	**Proof:** my car follows the object without zig-zagging
3	**Wireless link:** send motor commands from Python to an **ESP32** over **UDP**	**Proof:** my car responds within a blink of my laptop's command
4	**Fail safe:** add a **timeout** so the car stops if commands stop arriving	**Proof:** pulling the Wi-Fi makes the car stop within 0.5 s

LEARN

Objective 1 · Closed loop · video

Video time!



How do self-driving cars see?

TED-Ed · about 5 minutes · youtu.be/PRg5RNU_JLk

Watch for...

1. Which **sensors** do self-driving cars use?
2. Why do they combine **more than one** sensor?
3. What makes driving hard for a computer?

Scan to watch

After the video: Our car will use only a camera. What could go wrong that a lidar would catch?

LEARN

Objective 1 · Closed loop

Closed-loop control

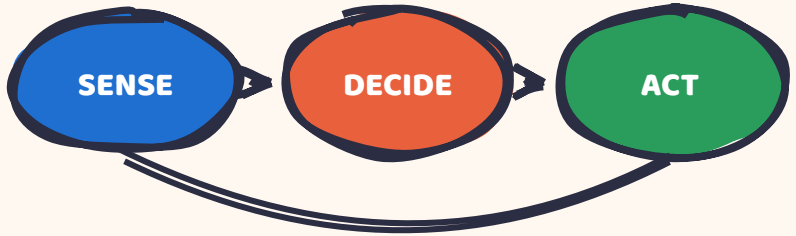
The idea

Open loop: 'drive forward 2 seconds' and hope. **Closed loop**: measure, correct, repeat.

Each frame: SENSE (find cx) → DECIDE (how far off?) → ACT (set motor speeds) → the world changes → SENSE again.

Error = cx − centre. Frame 640 px wide: centre = 320. Ball at cx = 420 → error = **+100** (right). Goal: error → 0.

Example: carrying a full cup of chai



You don't walk with your eyes closed. You watch the chai, notice it tilting, and correct your hand, again and again. That's a closed loop.

LEARN

Objective 2 · P-controller

Proportional (P) control

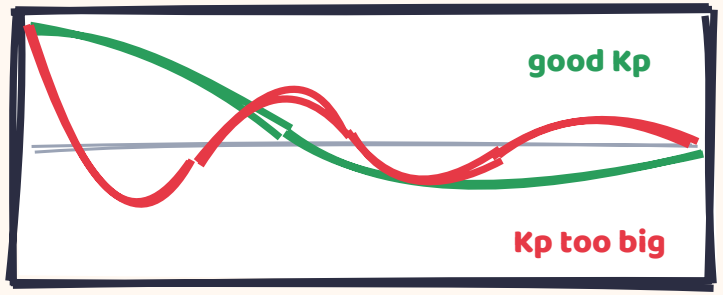
The idea

Turn **in proportion** to the error:
turn = Kp × error
left = base + turn, **right = base − turn**

Small error → gentle turn. Big error → sharp turn.

Kp too small: slow, lazy, loses the ball. **Kp** too big: overshoots and **oscillates** (zig-zag). Tune: start small, double until it wobbles, then back off by half.

Example: steering a bicycle



Riding towards a gate, you turn the handlebars a little if you're slightly off, more if you're far off. Jerk the bars fully every time and you'd wobble and fall.

Objective 3 · Wireless link

UDP: fast messages over Wi-Fi

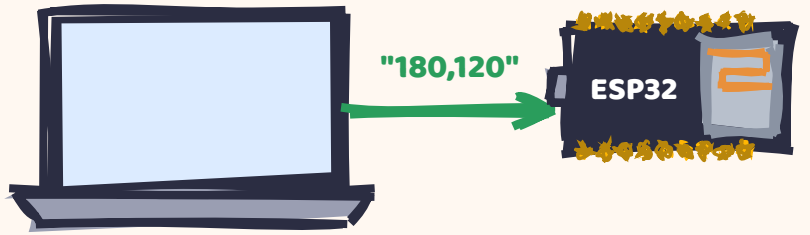
The idea

The laptop and ESP32 join the same Wi-Fi. Python sends tiny **UDP** packets like **"180,120"** (left, right speed) to the ESP32's IP address and port.

UDP is fire-and-forget: no waiting for 'got it', so it's **fast**. A lost packet doesn't matter; a newer one arrives 30 ms later. Perfect for live control (games use it too).

Total delay (camera + processing + Wi-Fi) is your **latency**: keep it under ~100 ms.

Example: shouting directions



A coach shouting 'left! right!' to a runner doesn't wait for a reply. If one shout is missed, the next one comes soon. That's UDP.

Objective 4 · Fail safe

Fail safe: stop when in doubt

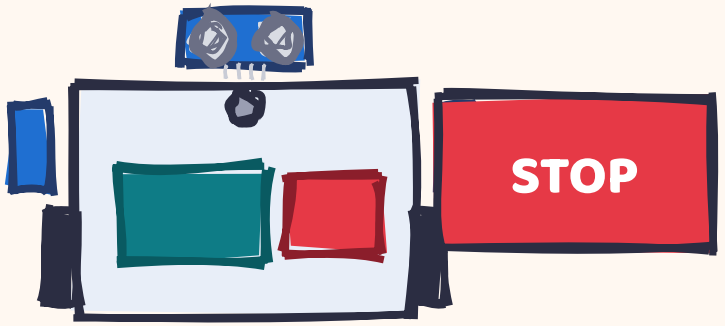
The idea

What if Wi-Fi drops, Python crashes, or the ball leaves the frame? A car that keeps its **last command** drives into a wall.

Watchdog on the ESP32: remember **millis()** of the last packet. No packet for **500 ms** → motors **off**.
In Python: ball not found → send **"0,0"** (or spin slowly to search).

Real robots are judged by how they **fail**, not just how they work.

Example: a dead man's switch



Train drivers must keep pressing a pedal. If they let go (they fall asleep), the train stops automatically. Safety by default.

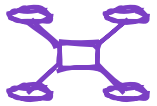
LEARN

Real world

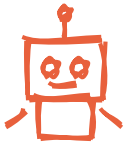
Vision + control in action



Lane keeping
Cars steer to keep the lane line centred: error → 0



Follow-me drones
Track a cyclist and keep them in frame



Warehouse robots
Follow floor markers and people safely



Farm robots
Steer along crop rows using cameras

Where could a follow-me robot help at school? What failure would be dangerous?

PID
Industry adds I (integral) and D (derivative) terms to P. You'll meet PID again in TL-25.

LEARN

Quick quiz

Control maths

Question	Answer
1. Width 640, cx = 250. Error?	
2. Kp = 0.5, error = −70. turn = ?	
3. base 150: left and right speeds?	
4. Car zig-zags. Change Kp how?	
5. Why UDP, not TCP, for steering?	

Answers

Cover! 1: −70 (left) · 2: −35 · 3: left 115, right 185 (turns left) · 4: make it smaller · 5: speed: old commands are useless, no waiting

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human P-controller
2. Our vision car kit
3. ESP32: UDP motor server + watchdog
4. Python: see, decide, send
5. Tune and test it
6. Tuning log
7. Control engineer challenges

Class game

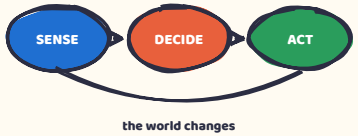
Human P-controller

How to play

1. One student is the **car**, eyes closed, walking slowly. One is the **controller**.
2. Target: a classmate holding a ball, who moves around the room.
3. Controller says only numbers: '−2' = small left, '+5' = big right, scaled by distance off.
4. Try Kp = 'tiny turns' vs 'huge turns'. Which reaches the target smoothly?

Then discuss

What happened with huge turns? With tiny turns? What's the best Kp for a human?



Big idea

Proportional = correct more when you're further off.

Build it

Our vision car kit



2WD car + L298N
From earlier projects



ESP32
Receives UDP, drives motors



Camera
Phone as IP cam or USB webcam on the car



Laptop
Python + OpenCV tracker (TL-17)



Hotspot
Laptop + ESP32 on the same network



Target
Bright ball on a stick

Camera options: a phone with an IP-camera app strapped to the car (cv2.VideoCapture('http://phone-ip:8080/video')), or a long USB webcam cable.

Test on the floor
Motors on a table = car off the edge. Test with wheels lifted first, then on the floor in a clear area.

Objective 3 · Wireless link · Build step 1 of 3

ESP32: UDP motor server + watchdog

```
#include <WiFi.h>
#include <WiFiUdp.h>
WiFiUDP udp;
const int ENA = 14, IN1 = 27, IN2 = 26, ENB = 25, IN3 = 33, IN4 = 32;
unsigned long lastPacket = 0;

void drive(int l, int r) {
  digitalWrite(IN1, l > 0); digitalWrite(IN2, l < 0);
  digitalWrite(IN3, r > 0); digitalWrite(IN4, r < 0);
  analogWrite(ENA, constrain(abs(l), 0, 255));
  analogWrite(ENB, constrain(abs(r), 0, 255));
}

void setup() {
  Serial.begin(115200);
  for (int p : {ENA, IN1, IN2, ENB, IN3, IN4}) pinMode(p, OUTPUT);
  WiFi.begin("ClassHotspot", "password");
  while (WiFi.status() != WL_CONNECTED) delay(200);
  Serial.println(WiFi.localIP());          // put this IP in Python
  udp.begin(4210);
}

void loop() {
  if (udp.parsePacket()) {
    char buf[32]; int n = udp.read(buf, 31); buf[n] = 0;
    int l, r;
    if (sscanf(buf, "%d,%d", &l, &r) == 2) { drive(l, r); lastPacket = millis(); }
  }
  if (millis() - lastPacket > 500) drive(0, 0);     // WATCHDOG
}
```

IP

Open Serial Monitor to read the ESP32's IP address.

Watchdog

No command for 500 ms = stop. Test it by stopping Python.

sscanf

Turns the text '180,120' into two ints.

Objective 2 · P-controller · Build step 2 of 3

Python: see, decide, send

```
import cv2, numpy as np, socket
ESP = ("192.168.4.23", 4210)                          # your ESP32's IP
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
LOWER, UPPER = np.array([5, 120, 80]), np.array([20, 255, 255])
KP, BASE, STOP_AREA = 0.4, 150, 25000
cap = cv2.VideoCapture(0)

while True:
    ok, frame = cap.read()
    if not ok: break
    frame = cv2.resize(frame, (640, 480))
    mask = cv2.inRange(cv2.cvtColor(frame, cv2.COLOR_BGR2HSV), LOWER, UPPER)
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    left = right = 0                                  # default: stop
    if cnts:
        c = max(cnts, key=cv2.contourArea); M = cv2.moments(c)
        if M["m00"] > 500:
            cx = M["m10"] / M["m00"]
            error = cx - 320                          # + means target is right
            turn = KP * error
            base = 0 if M["m00"] > STOP_AREA else BASE   # close enough? stop
            left, right = int(base + turn), int(base - turn)
            cv2.circle(frame, (int(cx), 240), 8, (0, 0, 255), -1)
    sock.sendto(f"{left},{right}".encode(), ESP)
    cv2.putText(frame, f"L {left} R {right}", (10, 30), 0, 1, (255, 255, 255), 2)
    cv2.imshow("pilot", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"): break
sock.sendto(b"0,0", ESP)
```

P-control

turn = KP × error. Start with KP = 0.2 and increase.

Stop area

When the target looks big (close), base = 0: the car only turns to face it.

Exit

Always send '0,0' when quitting.

Objective 2 · Build step 3 of 3

Tune and test it

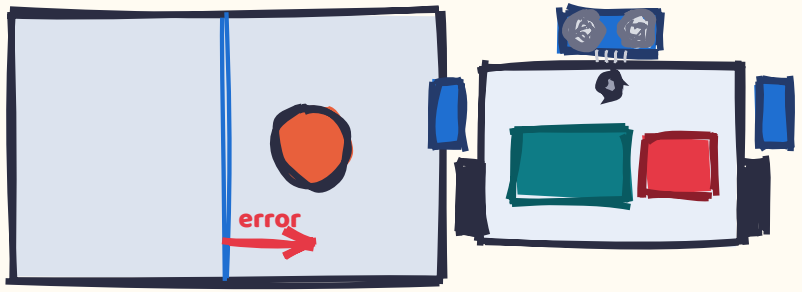
a Wheels **lifted**: move the ball left and right. Do the right wheels speed up/slow down correctly? If backwards, flip the sign of error.

b On the floor: KP = 0.2. Does it turn towards the ball but slowly?

c Double KP until it zig-zags. Then use about **half** of that value.

d Hide the ball: the car must stop. Stop Python: the watchdog must stop it.

e Measure latency: wave the ball and film with a phone in slow-mo. Count frames.



Check: Your car follows smoothly and stops when the target or link disappears.

Keep the test area clear. One person is always ready to pick the car up.

Test like an engineer

Tuning log

Test	Result	Notes
KP = 0.2: behaviour		
KP = 0.4: behaviour		
KP = 0.8: behaviour		
Watchdog stop time		ms
Measured latency		ms

Debug checklist

Turns the wrong way? Swap the sign of turn, or swap motor wires.
Car jerky? Camera FPS too low: shrink the frame.
No movement? Check the IP and that both are on the same Wi-Fi; Windows firewall may block UDP.

Write down every KP you try. Control engineering is careful experiments, not guessing.

MAKE

Level up!

Control engineer challenges

Choose your challenge



PD control: add Kd × (error − last_error) to damp the zig-zag.



Distance hold: use object area to keep a fixed distance (P control on speed too).



Search mode: if the target is lost, spin slowly until it's found again.



On-board: move tracking onto an ESP32-CAM with a tiny colour blob detector.

Think like an engineer

Why does more latency force you to use a smaller Kp?

Boss level

Make two cars: one follows the ball, the other follows the first car (preview of TL-26 swarms!).

Objective check

Can I really do it? Prove it.

01	Draw the sense → decide → act loop and say what 'error' is.	○ ○ ○
02	Show your car following. What Kp did you choose and how?	○ ○ ○
03	Show the UDP message format and your ESP32's IP.	○ ○ ○
04	Demonstrate the watchdog: stop Python. How fast does the car stop?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Follow-the-leader: the teacher walks with the ball; how long can each car keep it centred?

Show what you know

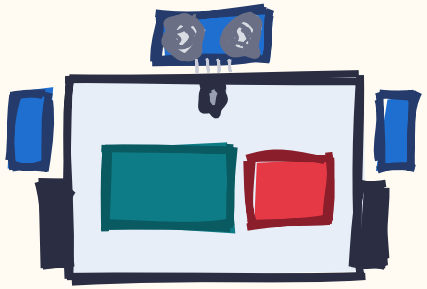
What did we learn?

- [x] Closed loops measure, correct and repeat
- [x] error = cx − centre; goal: error → 0
- [x] P-control: turn = Kp × error; too big = oscillation
- [x] UDP sends fast fire-and-forget commands
- [x] Watchdogs stop the robot when communication fails

Exit ticket: your car zig-zags left and right past the ball. What do you change first?

At home

Watch a driver keep a car in the lane. Notice the tiny corrections.



Next class:
TinyML Gesture Recogniser on ESP32!